A Beginner's Guide to Branching and Merging in FlutterFlow

What Section:

Welcome to the team! When working on a large app with multiple people, we don't all edit the exact same file at the same time. It would cause total chaos. Instead, we use a system called **Branching** and **Merging** (powered by Git behind the scenes).

Think of FlutterFlow projects as being made up of human-readable configuration files called **YAML** files. When we branch and merge, FlutterFlow is actually comparing these files to track changes.

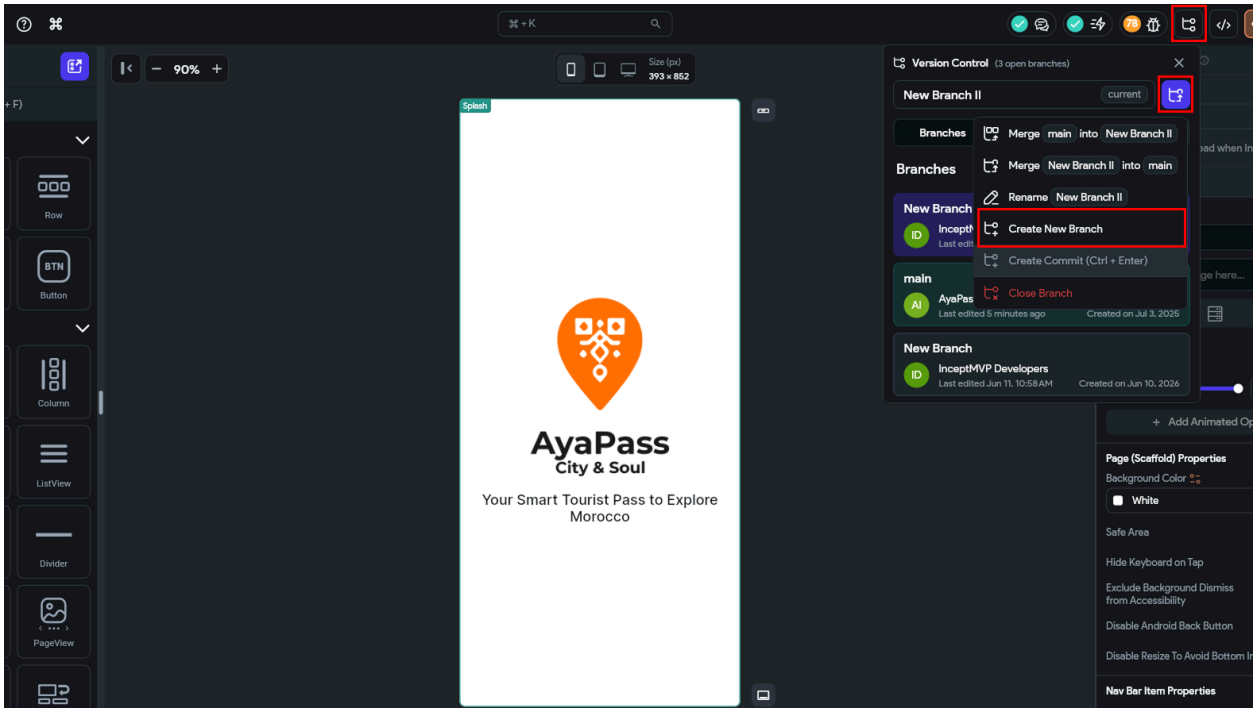
Here is everything you need to know to get started.

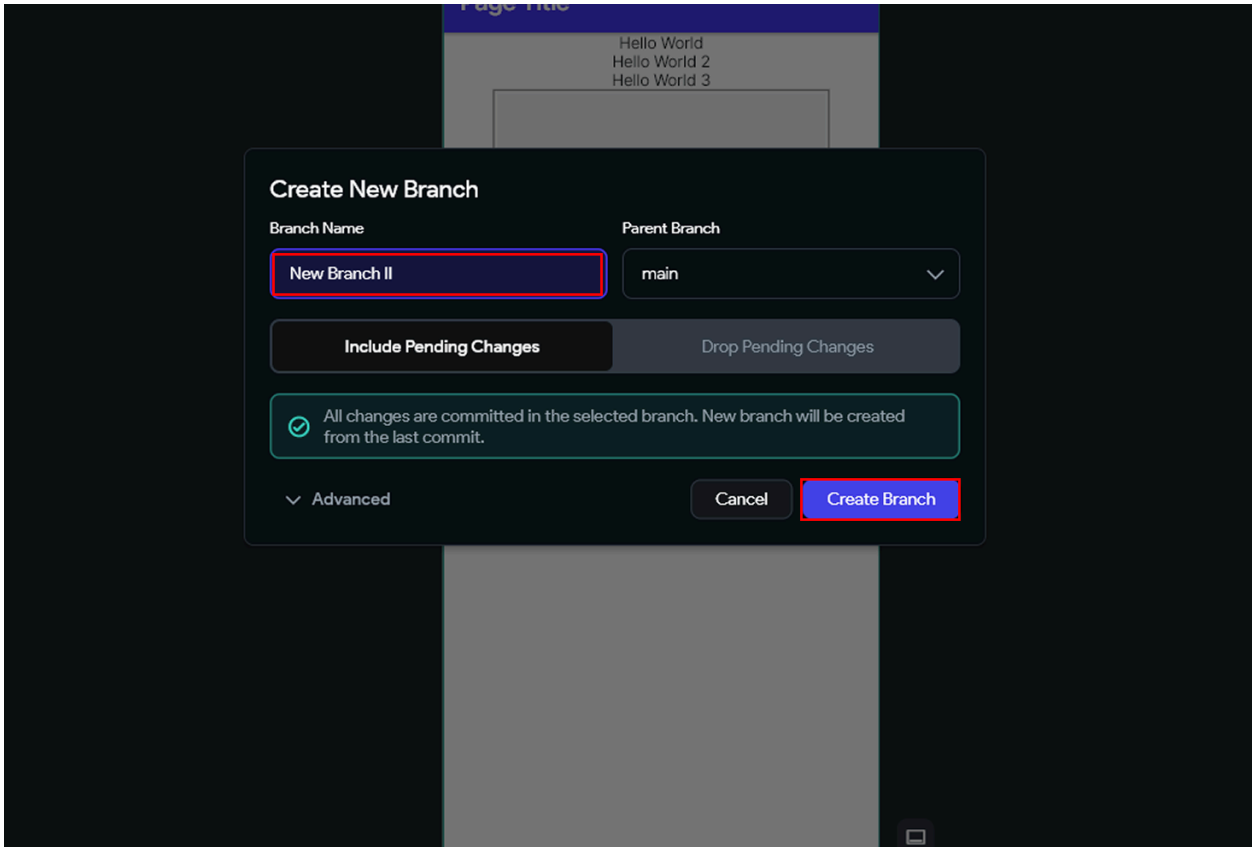
1. What is Branching?

Imagine you have a successful eCommerce app. You want to build a new "Product Recommendation" feature, but you don't want to risk breaking the live app while you are coding it.

Instead of working on the **Main** version, you create a **Branch**. A branch is like a safe, isolated copy of the project. You can add widgets, break things, and experiment without touching the live app or disrupting your teammates.

💡 **Tip:** Anyone can see the branching menu, but your team needs a **Growth Plan or higher** to create new branches. Also, branches created inside FlutterFlow stay inside FlutterFlow—they don't automatically sync to GitHub.





Why Section:

2. Why use Branching and Merging?

Using this system provides the safety and structure needed for professional development:

- Isolation: You can work on new features (e.g., "Product Recommendations") without risking the stability of the live eCommerce app.
- Collaboration: Multiple people can work on the same app simultaneously without overwriting each other's work.
- Auditability: Because of Commits, you can see exactly who changed what, when, and why.
- Conflict Prevention: By separating work into branches, you minimize the chance of two people editing the same item at the same time.

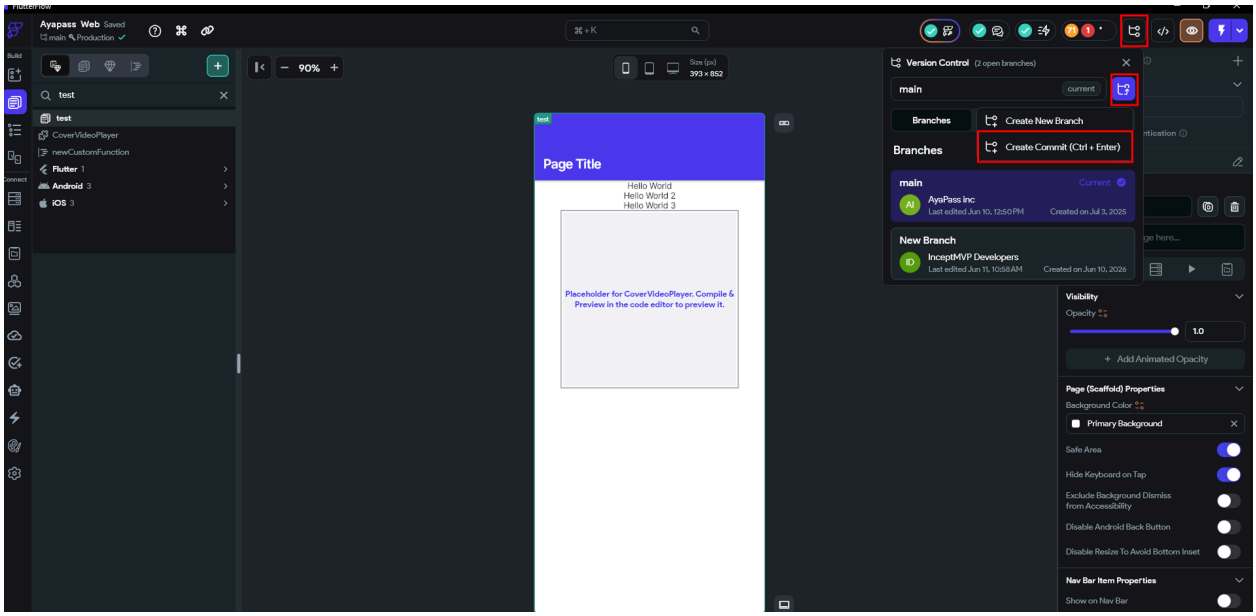
HOW Section:

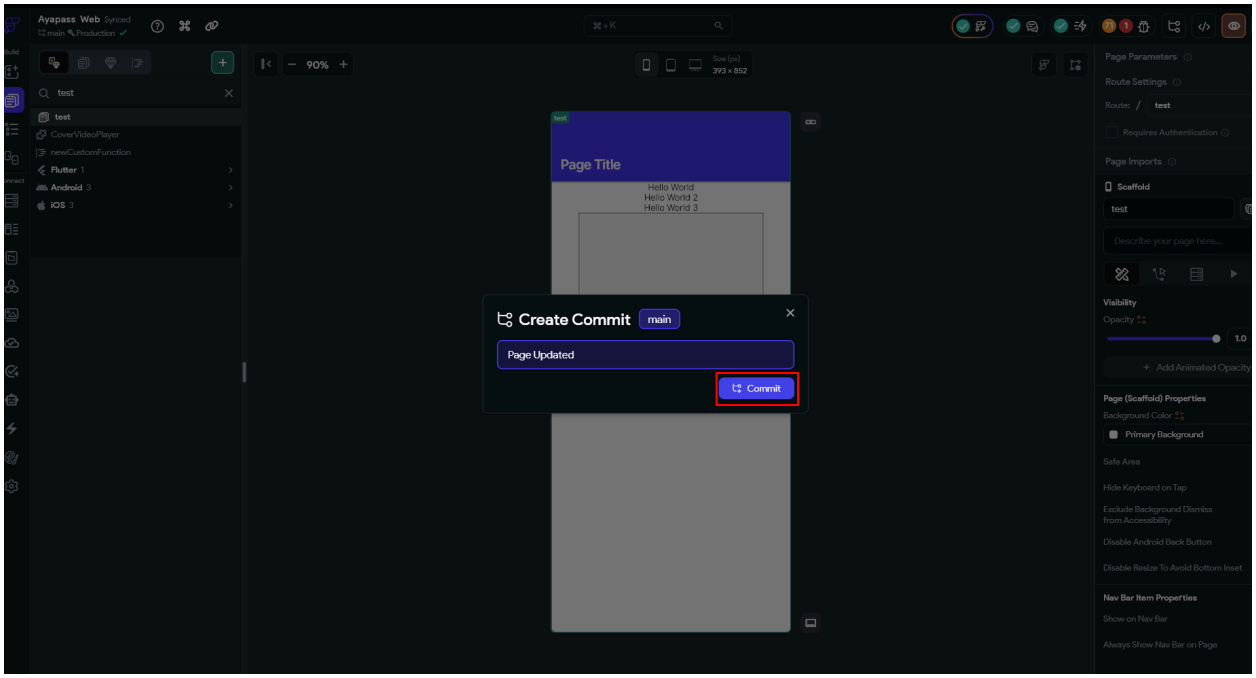
3. What is a Commit?

A **Commit** is like a manual "Save Game" checkpoint.
Whenever you finish a small task (like building a button, setting up an API call, or designing a page), you should create a commit. It saves a snapshot of your project at that exact moment.

Best Practices for Commits:

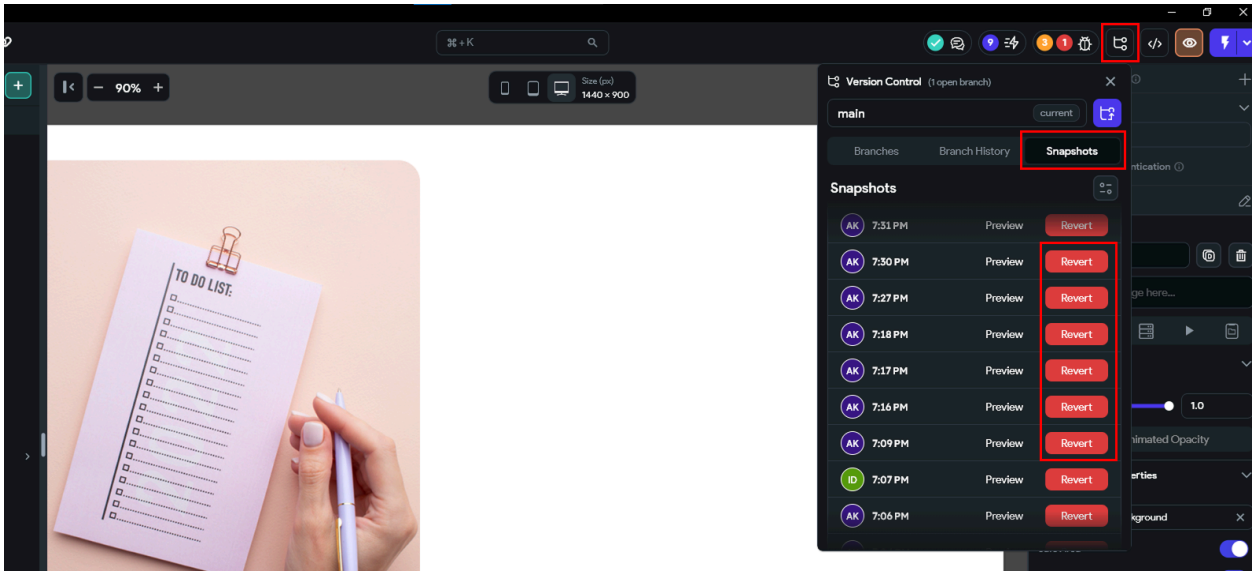
- **Commit Often:** Save your work frequently. You can use the shortcut `Cmd + Enter` (or `Ctrl + Enter` on Windows) to save quickly.
- **Write Clear Messages:** Don't just type "fixed stuff." Write "Changed submit button color to blue." Your future self and your teammates will thank you.
- **Test Before You Save:** Make sure your app actually runs before making a commit.





Commits vs. Snapshots vs. Versions

- **Snapshots:** Automatic backups FlutterFlow makes for you in the background.
- **Versions:** Manual checkpoints for the whole project.
- **Commits:** Advanced, flexible checkpoints used specifically inside branches. They let you see the exact lines of code that changed.



3. What is Merging?

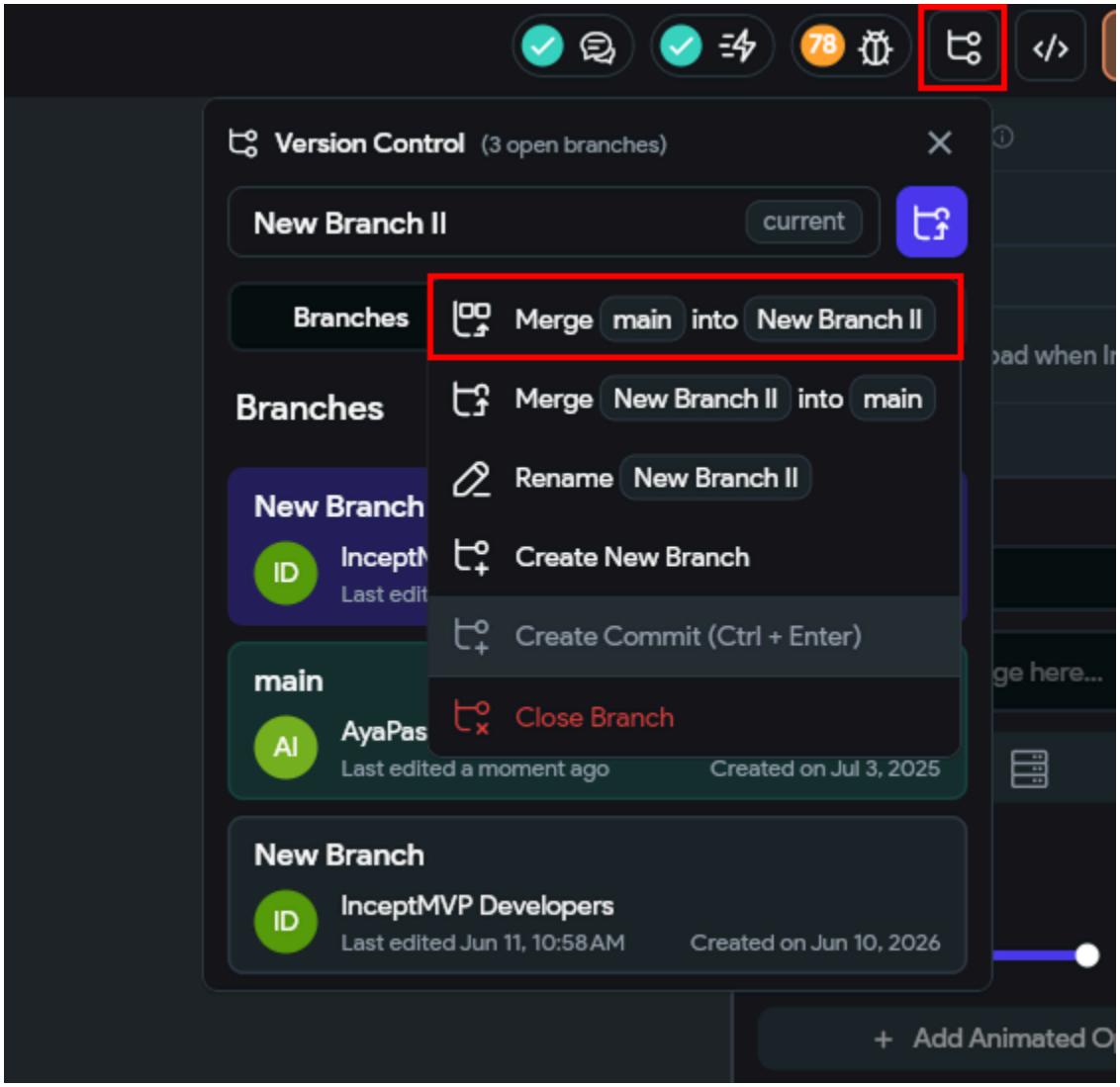
Once your new feature is finished and tested, you want to bring those changes back into the **Main** branch so everyone can use it. This process is called **Merging**.

Important Note on Merging: Merging does not just "mix" everything together. It compares the differences.

⚠️ **Warning:** If you completely delete a page in your feature branch and then merge it back to Main, Git thinks you *wanted* that page gone forever. It will delete it from Main too! Only make intentional, step-by-step changes.

Merging Directions:

1. **Parent → Child:** Pulling recent updates from the Main branch *down* into your feature branch to keep your workspace up-to-date.
2. **Child → Parent:** Pushing your finished feature branch *up* into the Main branch.



4. How to Handle Merge Conflicts

A **Merge Conflict** happens when two people change the exact same thing at the same time, and FlutterFlow doesn't know which version to keep.

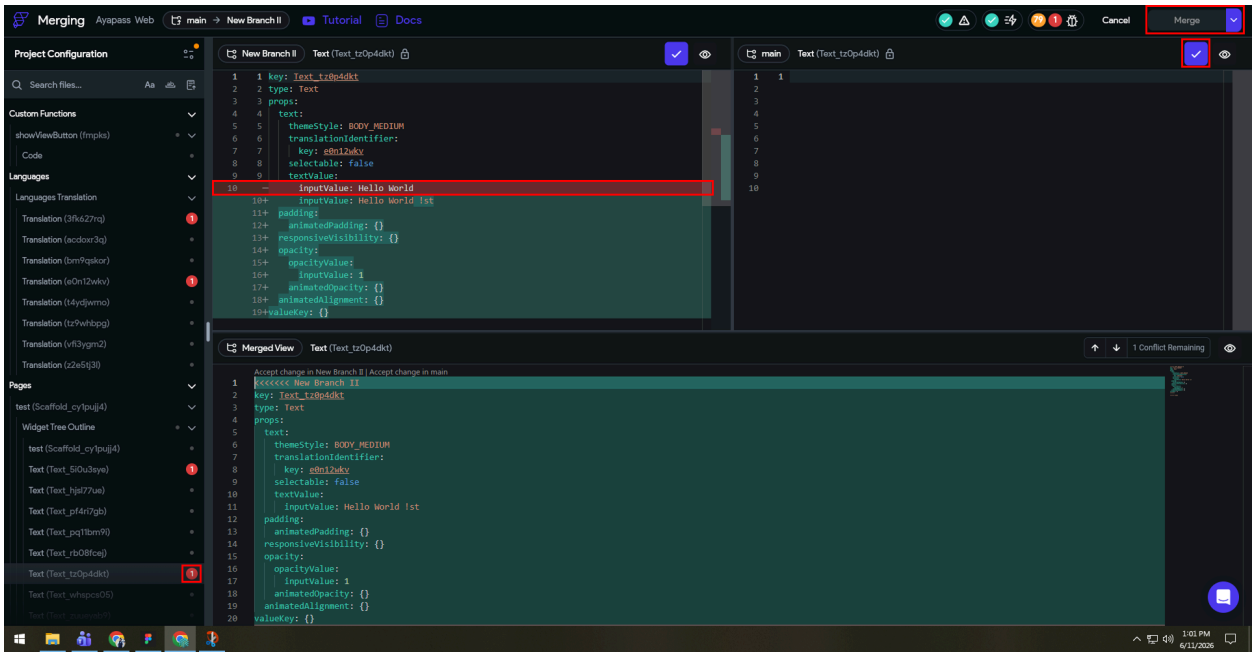
Example: > * **Alice** changes a button's text to "Submit Form" and makes it **Blue**.

- **Bob** changes the *same* button's text to "Send" and makes it **Green**.
- Alice merges her work first. When Bob tries to merge next, FlutterFlow will pause and ask: *"Hey, do you want Alice's blue button or Bob's green button?"*

Understanding the Merge Window

When a conflict happens, FlutterFlow will open a special merging screen:

- **The Left Panel:** Shows you a list of files. You can filter them to see only **"Files with Conflicts."**
- **The Right Upper Panel:** Shows a side-by-side comparison.
    - Line highlighted in **Red** = Content that was removed.
    - Line highlighted in **Green** = Content that was added.
    - You can click **"Accept Change"** to pick a side, or click the **Eye Icon** to see a visual preview of what the change looks like in the app builder.
- **The Right Lower Panel:** Shows the final combined file. If there is a conflict, you will see code markers like this:
    - `<<<<<<<` (The other branch's changes)
    - `=======` (The divider line)
    - `>>>>>>>` (Your branch's changes)
    - You can manually edit the text here to clean it up, then hit **Save Changes**.



Handling Errors During a Merge

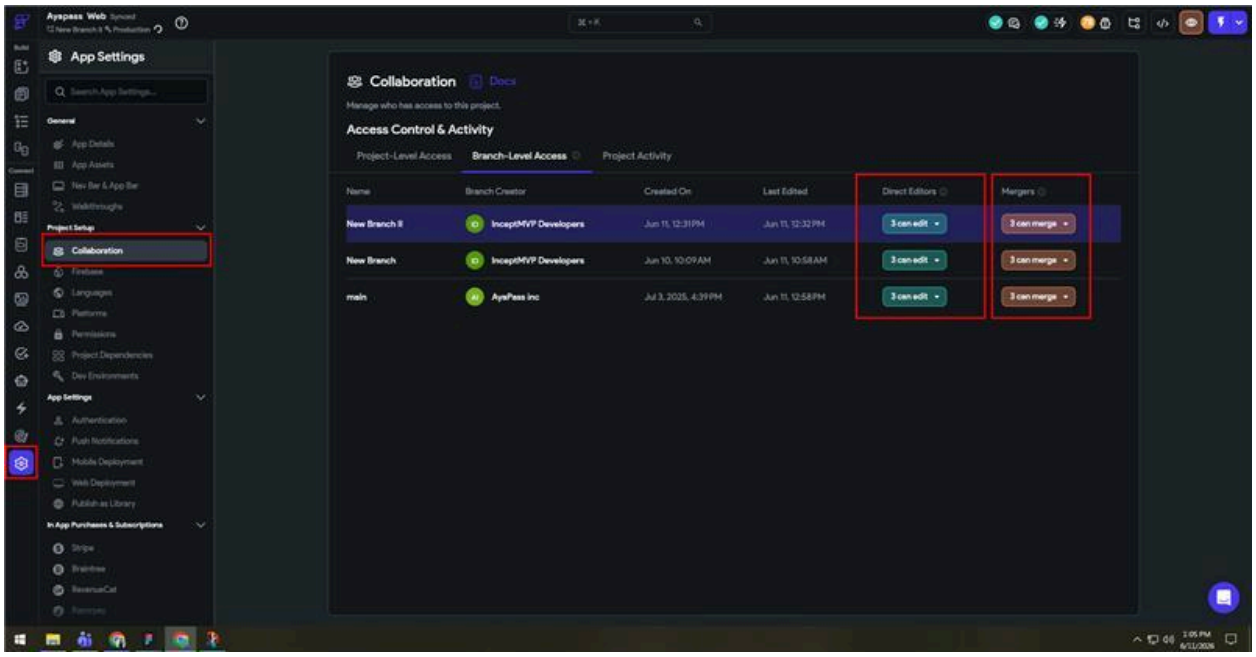
- **YAML Validation Errors:** This means the code format got messed up during the merge (e.g., a page was deleted in both branches, leaving 0 pages). You **must** fix these before you can finish the merge.
- **Project Errors:** This happens if the merge creates a logical error, like two data types having the exact same name. You can fix these *during* the merge, or finish the merge and fix them in the app builder afterward.

5. Branch Roles & Permissions

To keep our project safe, we can assign roles to team members for different branches (found under *Settings & Integrations* > *Project Setup* > *Collaboration*):

- **Editors:** Can work inside the branch and make direct changes.

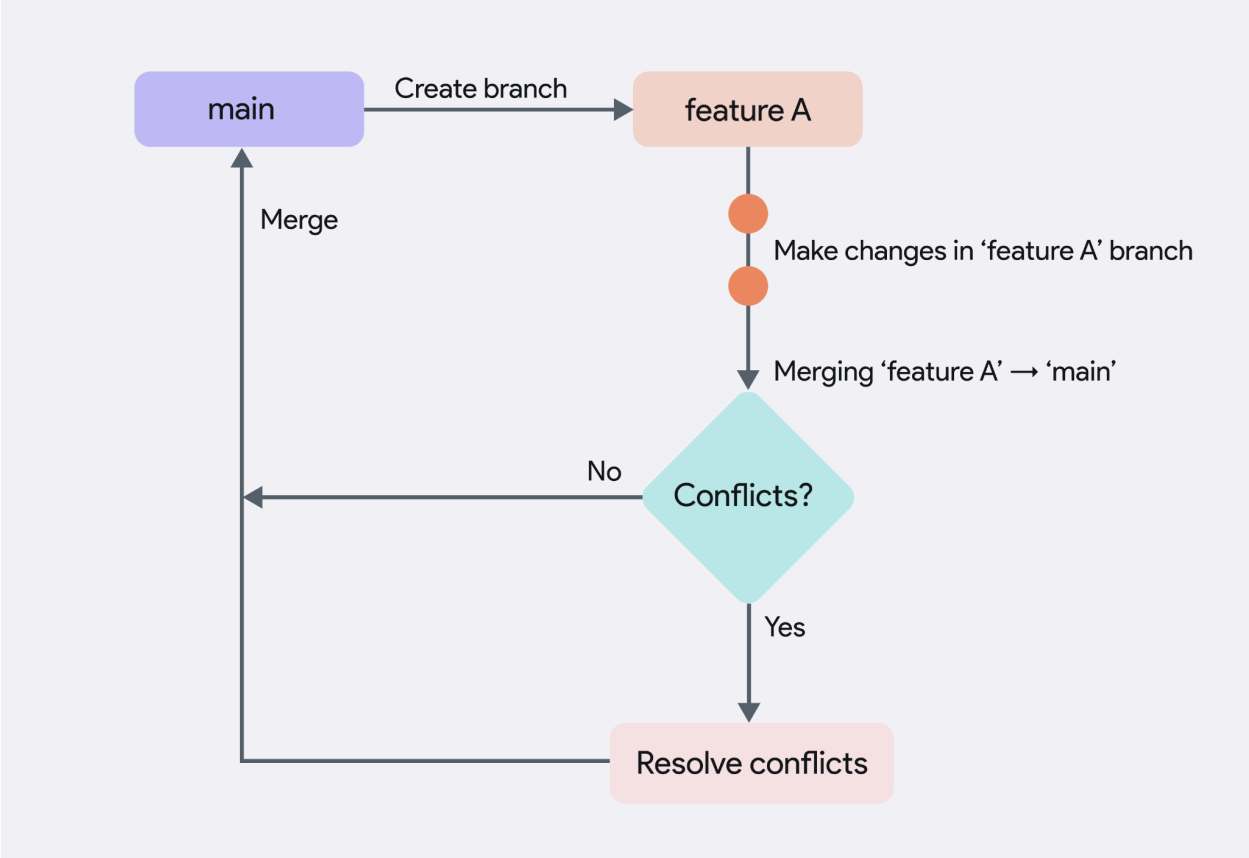
- **Mergers:** Can only merge other branches into this branch. We use this to protect the **Main** branch so nobody accidentally edits it directly.



6. Closing and Restoring Branches

Once your feature is safely merged into Main, the feature branch has done its job. To keep our workspace clean, you should **Close** the branch.

- Always talk to your team before closing a branch to ensure no one else is using it.
- If you accidentally close a branch, don't panic! You have **30 days** to go to the branch filter, find it under "Show Closed Branches," and click **Restore Branch**.


main
Create branch
feature A
Merge
Make changes in ‘feature A’ branch
Merging ‘feature A’ → ‘main’
No
Conflicts?
Yes
Resolve conflicts